correct OIDC login and CSRF context.

```
@Test
void whenLogoutAuthenticatedAndWithCsrfTokenThen302() {
  when(clientRegistrationRepository.findByRegistrationId("test"))
    .thenReturn(Mono.just(testClientRegistration()));

  webClient
    .mutateWith(
     SecurityMockServerConfigurers.mockOidcLogin())                ❶
    .mutateWith(SecurityMockServerConfigurers.csrf())              ❷
    .post()
    .uri("/logout")
    .exchange()
    .expectStatus().isFound();                                     ❸
}

private ClientRegistration testClientRegistration() {
  return ClientRegistration.withRegistrationId("test")             ❹
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
    .clientId("test")
    .authorizationUri("https://sso.polarbookshop.com/auth")
    .tokenUri("https://sso.polarbookshop.com/token")
    .redirectUri("https://polarbookshop.com")
    .build();
}
```

❶ Uses a mock ID Token to authenticate the user

❷ Enhances the request to provide the required CSRF token

❸ The response is a redirect to Keycloak to propagate the logout operation.

❹ A mock ClientRegistration used by Spring Security to get the URLs to contact Keycloak

Finally, run the tests as follows:

```
$ ./gradlew test --tests SecurityConfigTests
```

As always, you can find more test examples in the source code repository accompanying this book. When it comes to security, unit and integration tests are critical for ensuring the correctness of an application, but they are not enough. Those tests cover the default security configuration, which might be different in production. That’s why we also need security-oriented